

Project Requirement Document

ECommerce Product Management System

Angular + ASP.NET Core Microservices + Ocelot + SQL Server + EF Core + JWT

Document Type	Corporate Training Project Requirement Document
Primary Topic	Product lifecycle management for ecommerce catalog operations
Core Roles	Admin, Product Manager, Content Executive, Customer (preview)
Architecture	4 Microservices with Ocelot API Gateway
Quality Goal	JWT security, EF Core code-first, >=90% backend coverage

Prepared for structured project execution, architecture planning, UI wireframing, testing, and student deliverables.

A) Application Overview

The ECommerce Product Management System is designed to centralize the way an online business creates, enriches, approves, prices, publishes, and monitors products across its digital storefront. In many ecommerce teams, product data is scattered across spreadsheets, image folders, email approvals, and disconnected admin screens. This creates delays in product launches, inconsistent pricing, incorrect inventory visibility, and poor customer experience on the storefront.

This project solves that problem by introducing a structured Angular-based product management portal backed by ASP.NET Core microservices. The application gives administrators and product operations teams a controlled workflow for product creation, media upload, pricing, stock updates, approval, publishing, reporting, and audit tracking. A storefront preview is also included so the business can validate how a product will look before it is released to customers.

The system uses exactly four microservices with Ocelot API Gateway, JWT-based security, EF Core code-first persistence, and SQL Server. It is suitable for corporate training because it covers real-world product lifecycle management while also exposing students to enterprise architecture, API design, testing strategy, and documentation practices.

B) Objective

- Provide a fast and structured product onboarding experience for ecommerce operations teams.
- Enable secure role-based login and controlled access to product, pricing, inventory, and reporting features.
- Reduce product launch errors by validating mandatory catalog data, pricing rules, stock availability, and approval status before publishing.
- Design a scalable microservices architecture that separates identity, catalog, workflow, and reporting responsibilities.
- Support reporting, exports, and auditability so the business can monitor product quality and operational health.

C) Roles & Access Control

Business Roles

Role	Key Access	Typical Actions	Security Note
Customer (Preview User)	Read-only preview pages	View product listing, details, stock message, and purchase-facing presentation	Protected only where internal preview links are used
Admin	Full administrative control	Approve products, manage users, publish/unpublish items, bulk updates, dashboards, exports	Admin-only APIs via [Authorize(Roles="Admin")]
Product Manager	Catalog ownership	Create products, edit metadata, submit for approval, monitor quality issues	JWT role claim limits access to catalog and workflow functions
Content Executive	Data enrichment	Upload images, descriptions, attributes,	Restricted from final

		SEO text, and tags	approval/publish actions
--	--	--------------------	--------------------------

Security Implementation

- Angular route guards protect /auth/*, /customer/*, and /admin/* routes based on authenticated role and token presence.
- JWT Bearer authentication is used on all secured ASP.NET Core endpoints.
- Sensitive endpoints such as product approval, publish, bulk status change, and report exports are restricted to Admin role.
- DTO models are exchanged through APIs so EF Core entities are never exposed directly to the client.

D) UI Design Architecture (Angular)

The frontend is designed as an Angular single-page application with clear separation between reusable infrastructure and feature modules. The UI supports both product administration and storefront preview workflows.

- CoreModule - application shell, interceptors, guards, environment configuration, and global services.
- SharedModule - reusable cards, badges, modals, tables, form controls, validation summaries, spinners, and toast notifications.
- AuthModule - signup, login, forgot password, token refresh handling, and unauthorized screen.
- CustomerModule - product listing, product details, preview cart flow, and product history preview.
- AdminModule - dashboard, product CRUD, pricing, inventory, approval workflow, bulk upload, audit views.
- ReportsModule (optional) - charts, exports, KPI widgets, and operational reports.

Routing follows /auth/*, /customer/*, and /admin/* paths. A JWT interceptor appends bearer tokens to protected requests, while a global error handler converts backend failures into standardized notifications. Reusable components include product cards, filter sidebar, pagination, image gallery, stock badge, publish status badge, wizard footer, confirmation dialog, and alert banner.

UI Design Pattern (1 pattern)

Chosen Pattern: Wizard / Stepper Pattern. This pattern is applied to the product publish flow because product onboarding typically happens in stages. The stepper guides the user through Basic Information -> Media -> Pricing -> Inventory -> Review -> Publish. It prevents incomplete product launches, surfaces validation messages at the correct step, and creates a more teachable UI flow for students.

E) Page-by-Page Functional Requirements

1. Landing / Home

- Purpose - Present the overall product management console and operational KPIs.
- UI Components - Dashboard cards, quick links, alert panel, recent products, left navigation.
- Fields + Validations - No direct form fields; filters may include date range and business unit.
- API Calls - GET dashboard summary, GET pending alerts via Admin/Reporting service through /gateway/admin/*.
- Success / Error Messages - Show success tiles on successful load; show retry banner on API failure.
- Navigation Rules - Unauthenticated users are redirected to Login; authenticated admins land here after login.

2. Signup

- Purpose - Create a new internal product operations user.
- UI Components - Registration form, role dropdown, terms checkbox, password rules panel.

- Fields + Validations - Name required; email format; password complexity; confirm password match.
- API Calls - POST signup via Identity/Auth service.
- Success / Error Messages - Display account created message or duplicate email error.
- Navigation Rules - After success navigate to Login.

3. Login

- Purpose - Authenticate the user and issue JWT token.
- UI Components - Email field, password field, remember me, login button.
- Fields + Validations - Required email/password; lockout message after configured failures.
- API Calls - POST login via Identity/Auth service.
- Success / Error Messages - Show login success, invalid credential error, inactive user error.
- Navigation Rules - Route based on role to /admin/dashboard or permitted module.

4. Product Listing (PLP)

- Purpose - Allow users to search, filter, sort, and manage product catalog entries.
- UI Components - Search box, filter sidebar, table/cards, status chips, bulk selection.
- Fields + Validations - Search text optional; filter values controlled; bulk action requires selected rows.
- API Calls - GET products, GET categories, GET search facets via Catalog service.
- Success / Error Messages - No records message, filter applied message, service unavailable banner.
- Navigation Rules - Clicking a row navigates to Product Details/Edit.

5. Product Details

- Purpose - View and edit a product including metadata, images, and variants.
- UI Components - Image gallery, tabbed sections, basic info form, attribute matrix, save buttons.
- Fields + Validations - Product name, SKU, category required; image type validation; attribute constraints.
- API Calls - GET product detail, PUT product update, POST media upload.
- Success / Error Messages - Save success toast, duplicate SKU error, file upload error.
- Navigation Rules - Back to listing, next to Pricing/Inventory step.

6. Cart / Storefront Preview

- Purpose - Show how the product appears in a customer-facing shopping context.
- UI Components - Preview card, price snapshot, stock message, CTA area, validation notes.
- Fields + Validations - Read-only preview; system validates whether required publish fields exist.
- API Calls - GET preview model via Catalog service.
- Success / Error Messages - Display preview generated or missing field alerts.
- Navigation Rules - Return to Product Details or launch publish wizard.

7. Checkout / Publish Wizard

- Purpose - Guide the business user through publish readiness verification.
- UI Components - Stepper header, section forms, review card, completeness score.
- Fields + Validations - Mandatory checks for media, pricing, tax, inventory, and description completeness.
- API Calls - GET publish checklist, PUT step data, POST submit for approval via Product Workflow service.
- Success / Error Messages - Show step save success, incomplete step error, conflict warning.
- Navigation Rules - Next/previous navigation inside wizard; finish submits for approval.

8. Payment (integration stub) / Pricing Approval

- Purpose - Simulate external dependency behavior for training purposes.
- UI Components - Approval stub panel, pricing confirmation, audit comments.
- Fields + Validations - Approval remark required on reject; price range rules enforced.

- API Calls - POST pricing approval simulation via Product Workflow service.
- Success / Error Messages - Approved/rejected messages and audit trail entry.
- Navigation Rules - On success return to workflow summary.

9. Order Confirmation / Publish Confirmation

- Purpose - Confirm the product has been approved or published.
- UI Components - Summary card, product snapshot, timestamp, next actions.
- Fields + Validations - No input; acknowledgment only.
- API Calls - POST final publish or GET confirmation details via Product Workflow service.
- Success / Error Messages - Publish success message or downstream failure rollback message.
- Navigation Rules - Navigate to Product History or Dashboard.

10. Order History + Order Details / Product Audit History

- Purpose - Track all changes made to the product.
- UI Components - Audit table, filters, detail side panel, before/after view.
- Fields + Validations - Date filters optional; export format required when exporting.
- API Calls - GET audit history and GET audit detail via Admin/Reporting service.
- Success / Error Messages - No history message or download successful message.
- Navigation Rules - Drill into specific entry details.

11. Admin Dashboard

- Purpose - Give leadership view of catalog and workflow health.
- UI Components - KPI cards, charts, approval queue, low stock alerts, publish trend.
- Fields + Validations - Filter by date, category, brand, owner.
- API Calls - GET reporting summary via Admin/Reporting service.
- Success / Error Messages - Load success, empty state, partial data warning.
- Navigation Rules - Links to product management, approvals, reports.

12. Admin Product Management (CRUD)

- Purpose - Support create, read, update, delete, clone, bulk upload, and archive actions.
- UI Components - Toolbar, editable grid, modal dialogs, import/export panel.
- Fields + Validations - Mandatory product identity fields; delete requires confirmation; import file type checks.
- API Calls - POST/PUT/DELETE via Catalog service and Product Workflow service.
- Success / Error Messages - Saved, archived, imported, or conflict error messages.
- Navigation Rules - Action buttons vary by role and status.

13. Admin Orders Management / Product Status Management

- Purpose - Manage lifecycle states such as Draft, In Review, Approved, Published, Rejected, Archived.
- UI Components - Queue table, detail panel, status action buttons, remarks modal.
- Fields + Validations - Reject/archive requires reason; publish requires checklist pass.
- API Calls - PUT status update via Product Workflow service.
- Success / Error Messages - Status changed message or forbidden 403 message.
- Navigation Rules - Return to dashboard queue or product detail.

14. Admin Reports

- Purpose - Generate reports on price changes, product readiness, low stock, and catalog quality.
- UI Components - Report filters, chart area, export buttons.
- Fields + Validations - At least one filter required for large export jobs.
- API Calls - GET report data, POST export request via Admin/Reporting service.

- Success / Error Messages - Export ready message or timeout warning.
- Navigation Rules - Remain on same page after export request.

F) Status Lifecycle

For this project, the main lifecycle is the product publish lifecycle. A clean status flow reduces confusion and ensures only validated products reach the storefront.

Status	Meaning	Who Can Change	Typical Transition
Draft	Product record exists but is incomplete.	Product Manager / Content Executive	Created -> In Enrichment
In Enrichment	Images, attributes, SEO, pricing, and stock are being added.	Product Manager / Content Executive	Draft -> In Enrichment -> Ready for Review
Ready for Review	Mandatory fields are complete and waiting for approval.	Product Manager	In Enrichment -> Ready for Review
Approved	Business validation completed successfully.	Admin	Ready for Review -> Approved
Published	Product is visible on storefront read model.	Admin	Approved -> Published
Rejected	Review failed; corrections are needed.	Admin	Ready for Review -> Rejected -> In Enrichment
Archived	Product removed from active operational use.	Admin	Published/Rejected -> Archived

Optional training branch: Published products can later enter Price Change Pending or Stock Hold as future enhancements. Students may mention future async event-driven updates for these extended states.

G) Microservices System Architecture (4 Services + Ocelot)

The application uses exactly four microservices. Inter-service communication begins with REST APIs for clarity and easier testing. Future enhancement can introduce asynchronous events for publish notifications, stock synchronization, and pricing updates.

- 1) Identity/Auth Service - signup, login, JWT generation, claims/roles, user status management.
- 2) Catalog Service - products, categories, attributes, media metadata, search, storefront read model.
- 3) Product Workflow Service - pricing, inventory, approval workflow, publish readiness, status management.
- 4) Admin/Reporting Service - dashboard aggregation, reports, exports, audit logs, operational alerts.
- Ocelot API Gateway routes all frontend traffic to the correct downstream service using /gateway/auth/*, /gateway/catalog/*, /gateway/workflow/*, and /gateway/admin/* paths.

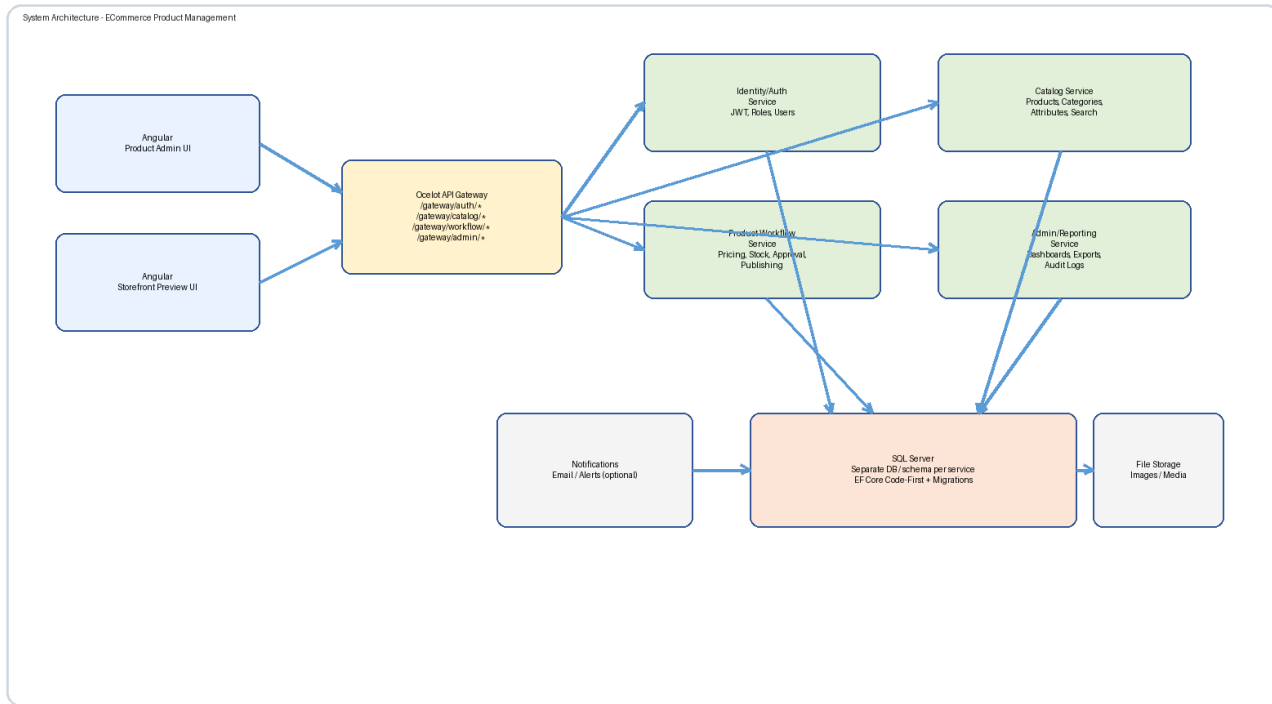


Figure G1. High-level architecture showing Angular UIs, Ocelot gateway, four microservices, SQL Server, optional notifications, and file storage.

H) Database Design (SQL Server + EF Core Code-First)

Each microservice owns its entity models, DbContext, migrations, and repository/service layers. SQL Server is the primary relational database. Students must use EF Core Code-First only.

- Suggested entities: Users, Roles, Categories, Products, ProductVariants, Prices, Inventory, MediaAssets, Approvals, AuditLogs.
- Relationships: Category 1..* Products, Product 1..* Variants, Variant 1..1 Inventory, Product 1..* MediaAssets, Product 1..* Approvals.
- Code-first flow: Create entity models -> configure DbContext -> add migrations -> update database -> verify schema in SSMS.
- Note: ER diagram below is a sample; students can design their own variant as long as it supports the required workflow.
- Note: DTO models should be used for data transactions; EF entities should not be exposed directly.

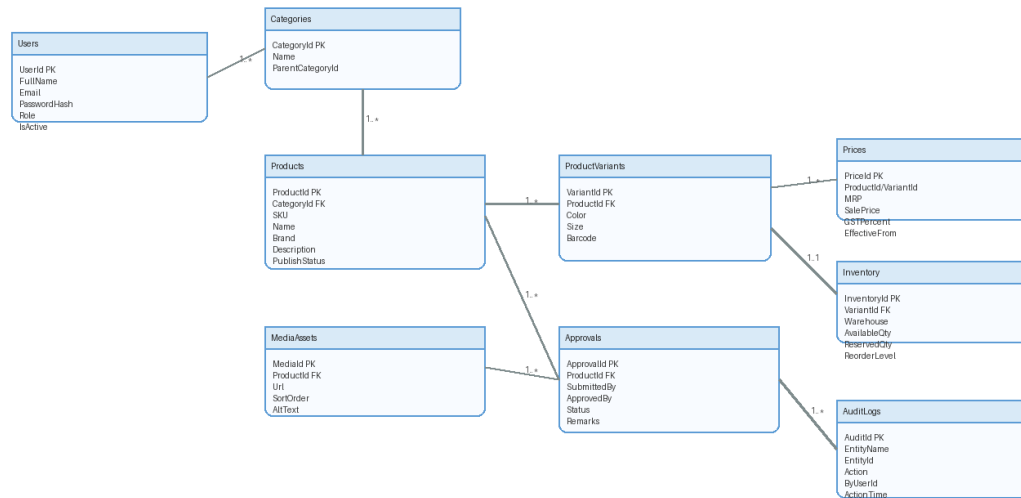


Figure H1. Sample ER diagram for the product management domain.

I) REST API Summary (By Service + Gateway)

Postman and Swagger/OpenAPI should be used for endpoint testing and documentation.

Service	Method + Route	Purpose	Gateway Example
Identity/Auth	POST /api/auth/signup	Create user account	/gateway/auth/signup
Identity/Auth	POST /api/auth/login	Validate credentials and issue JWT	/gateway/auth/login
Catalog	GET /api/products	List/search products	/gateway/catalog/products
Catalog	GET /api/products/{id}	Fetch product detail	/gateway/catalog/products/{id}
Catalog	POST /api/products	Create a product	/gateway/catalog/products
Catalog	PUT /api/products/{id}	Update product metadata	/gateway/catalog/products/{id}
Product Workflow	PUT /api/workflow/products/{id}/pricing	Save or update pricing	/gateway/workflow/products/{id}/pricing
Product Workflow	PUT /api/workflow/products/{id}/inventory	Save stock data	/gateway/workflow/products/{id}/inventory

Product Workflow	POST /api/workflow/products/{id}/submit	Submit product for review	/gateway/workflow/products/{id}/submit
Product Workflow	PUT /api/workflow/products/{id}/status	Approve/reject/publish/archive	/gateway/workflow/products/{id}/status
Admin/Reporting	GET /api/reports/dashboard	Load dashboard KPIs	/gateway/admin/reports/dashboard
Admin/Reporting	GET /api/audit/products/{id}	Load product audit history	/gateway/admin/audit/products/{id}

J) Software Required

- Angular CLI and Node.js LTS
- Visual Studio 2022 and/or Visual Studio Code
- .NET SDK compatible with ASP.NET Core target version
- Microsoft SQL Server Developer Edition and SQL Server Management Studio (SSMS)
- Ocelot NuGet package for API Gateway
- Postman and Swagger/OpenAPI for API testing
- Git, GitHub/Azure Repos, and package managers (npm / NuGet)

K) Testing & Quality Plan

Backend - NUnit

- Unit tests for service classes, validation rules, repositories, and workflow transitions.
- Target backend code coverage of at least 90% using coverlet and reportgenerator.
- Test JWT issuance, role validation, pricing rules, duplicate SKU checks, and publish readiness logic.

Frontend - Jasmine + Karma

- Component tests for product listing, product details, wizard step validation, and dashboard cards.
- Service tests with mocked HTTP calls.
- Guard tests for role-based routes such as /admin/* and preview access.

Quality / Vulnerability

- Use SonarQube if available; otherwise use SonarLint, dotnet analyzers, and npm audit.
- Check for package vulnerabilities and maintain coding standards.
- Add practical static checks such as nullable reference analysis, API contract validation, and DTO mapping reviews.

L) Student Deliverables (Diagrams + Planning)

- Use Case Diagram
- Class Diagram
- Sequence Diagram for key flow: Add Product -> Enrich -> Submit -> Approve -> Publish
- ER Diagram

- Work Breakdown Structure (WBS)
 - Agile sprint plan in Azure Boards / Jira / Trello
 - Sprint-wise backlog with story points or effort hours, sprint goal, and Definition of Done
- Recommended sprint model: 2-week sprints. Suggested Sprint 1 covers authentication and catalog base. Sprint 2 covers pricing, inventory, and workflow. Sprint 3 covers reporting, testing, and hardening.

M) Test Cases (Table Format)

Test Case	Role	Steps	Expected Result	Type	Related Service
TC01 Login Success	Admin	Enter valid email and password	JWT token issued and dashboard opens	Positive	Identity/Auth
TC02 Login Failure	Admin	Enter wrong password	Error message shown; no token issued	Negative	Identity/Auth
TC03 Search Product	Product Manager	Search by SKU/name	Matching products listed	Functional	Catalog
TC04 Create Product	Product Manager	Enter mandatory product data and save	Product stored in Draft state	Functional	Catalog
TC05 Duplicate SKU	Product Manager	Create another product with same SKU	Validation error blocks save	Negative	Catalog
TC06 Upload Invalid Image	Content Executive	Upload unsupported file type	File rejected with message	Negative	Catalog
TC07 Save Pricing	Product Manager	Enter valid MRP and sale price	Pricing saved and audit captured	Functional	Product Workflow

Table M1. Core authentication, catalog, pricing, and submission test cases.

M) Test Cases (Table Format) - Continued

Test Case	Role	Steps	Expected Result	Type	Related Service
TC08 Invalid Price Rule	Product Manager	Enter sale price higher than MRP	Validation error displayed	Negative	Product Workflow
TC09 Submit for Review	Product Manager	Complete checklist and submit	Status becomes Ready for Review	Functional	Product Workflow
TC10 Admin Approves Product	Admin	Open approval queue and approve item	Status becomes Approved	Functional	Product Workflow
TC11 Publish Product	Admin	Publish approved product	Product visible in storefront preview	Integration	Product Workflow/Catalog
TC12 Forbidden Access	Content Executive	Attempt admin-only publish endpoint	403 Forbidden returned	Security	Product Workflow
TC13 Dashboard Export	Admin	Run report and export data	Export file generated	Functional	Admin/Reporting
TC14 Audit History Load	Admin	Open product audit tab	All changes load in chronological order	Functional	Admin/Reporting

Table M2. Approval, security, reporting, and audit test cases.

N) Deliverables & Evaluation Criteria

Evaluation Area	Expectation	Weightage
Microservices + Gateway	Exactly four services with working Ocelot routes	20%
JWT Security	Role-based access, guards, and secured APIs	15%
EF Core Code-First	Entities, DbContext, migrations, repository/service usage	15%
UI vs Wireframes	Angular screens aligned with documented wireframes	15%
Testing Coverage	NUnit, Jasmine/Karma,	15%

	meaningful coverage	
Quality Gates	Static analysis, vulnerability checks, clean coding	10%
Diagrams + Sprint Plan	Complete diagrams, WBS, sprint backlog and DoD	10%

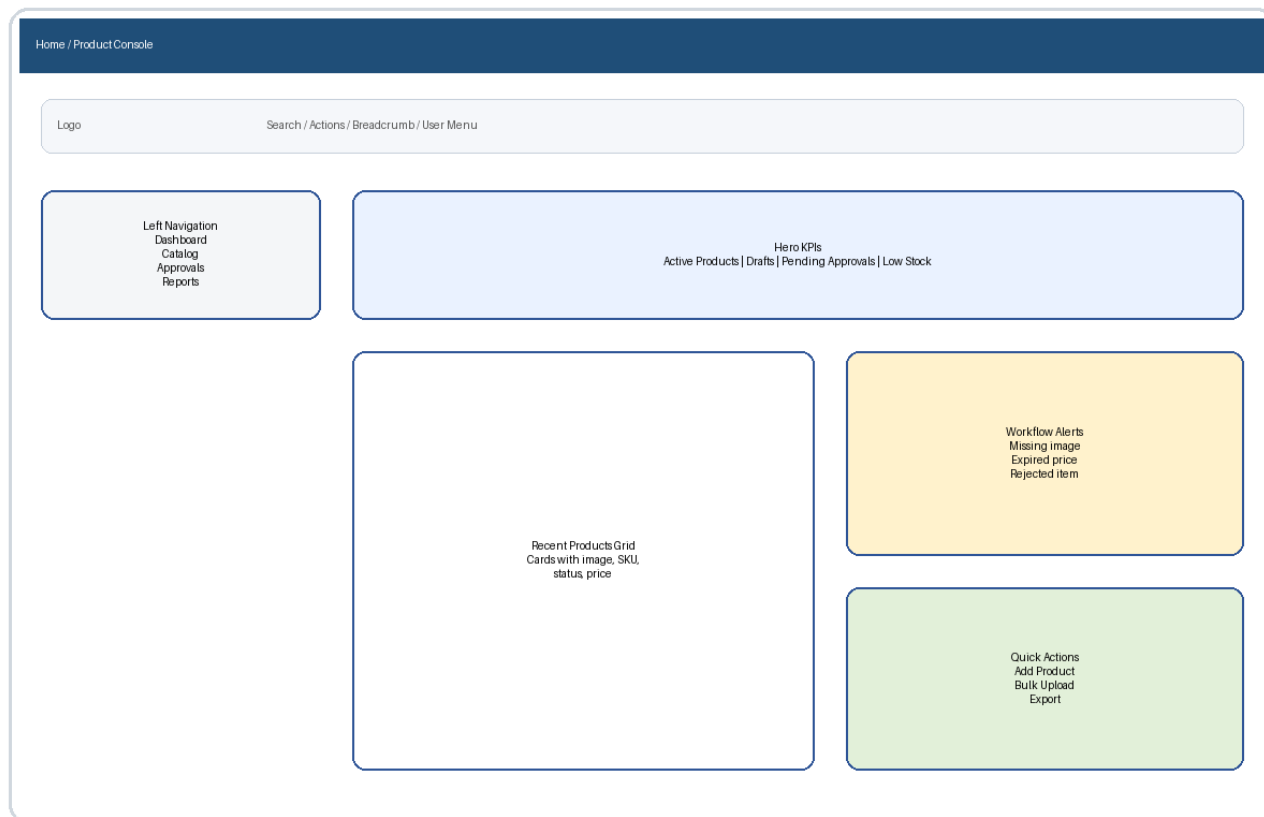
O) Conclusion

The ECommerce Product Management System is a strong real-world training project because it goes beyond simple CRUD and introduces the actual business lifecycle of catalog operations. Students learn how product information flows from creation to publication, how microservices divide responsibilities, how JWT secures enterprise applications, and how testing and quality practices support maintainable delivery. The project is large enough for architecture, UI, API, database, and agile planning exercises while still being realistic for guided implementation.

? Screen Design (Wireframes)

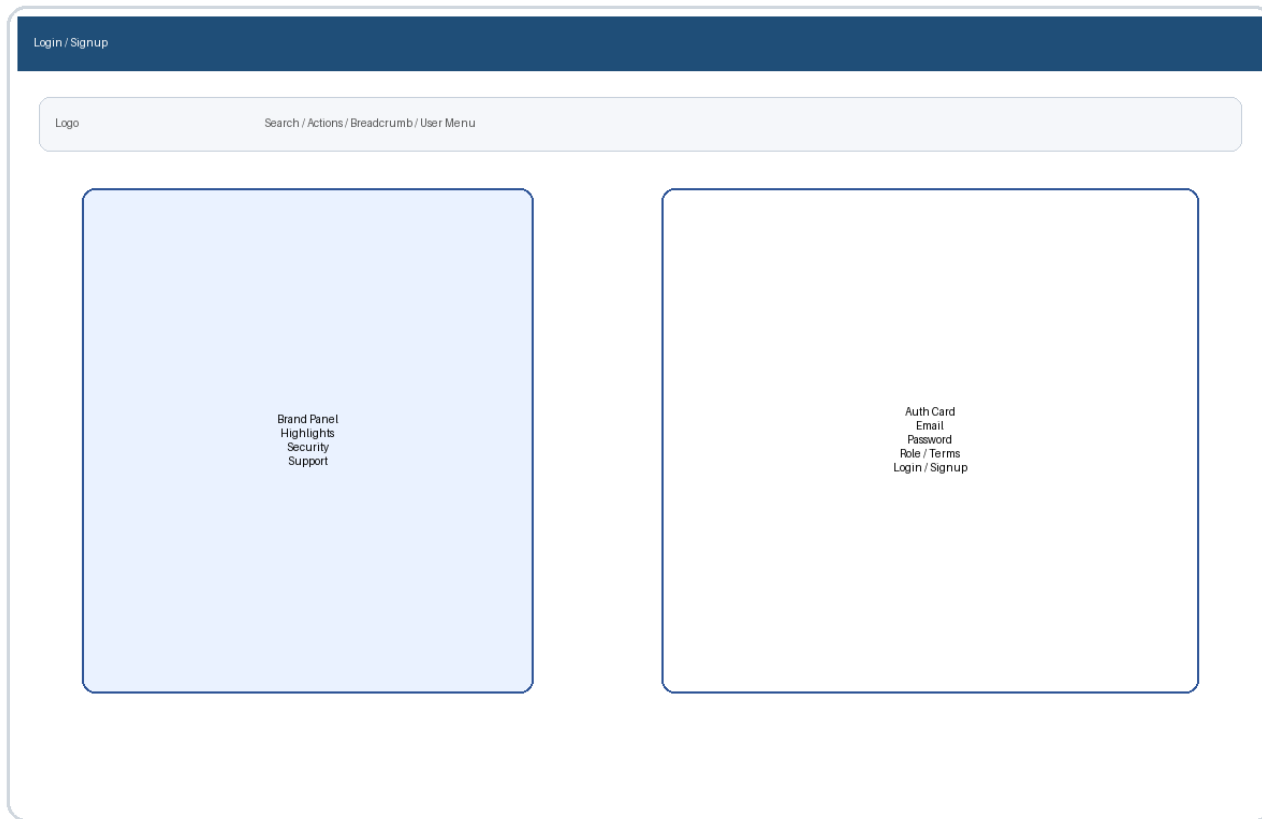
The following wireframes provide a clean reference for major screens. They are intentionally simple and aligned so students can translate them into Angular components without losing layout consistency.

Home Product Console



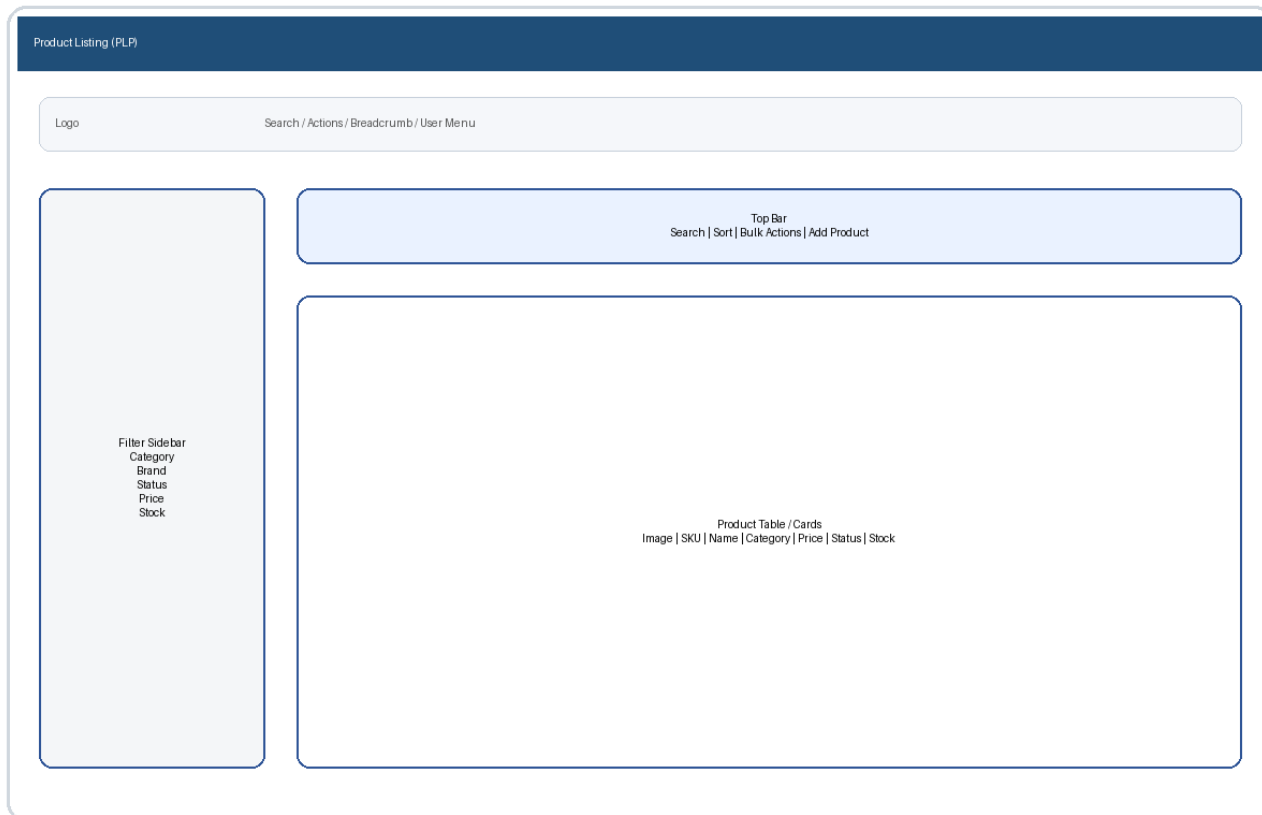
Wireframe reference - Home Product Console

Login Signup



Wireframe reference - Login Signup

Product Listing PLP



Wireframe reference - Product Listing PLP

Product Details Edit

Product Details / Edit

Logo

Search / Actions / Breadcrumb / User Menu

Image Gallery
Main image
Thumbnails
Upload

Basic Info
Name
SKU
Brand
Description
Category

Variant Matrix
Color / Size / Barcode

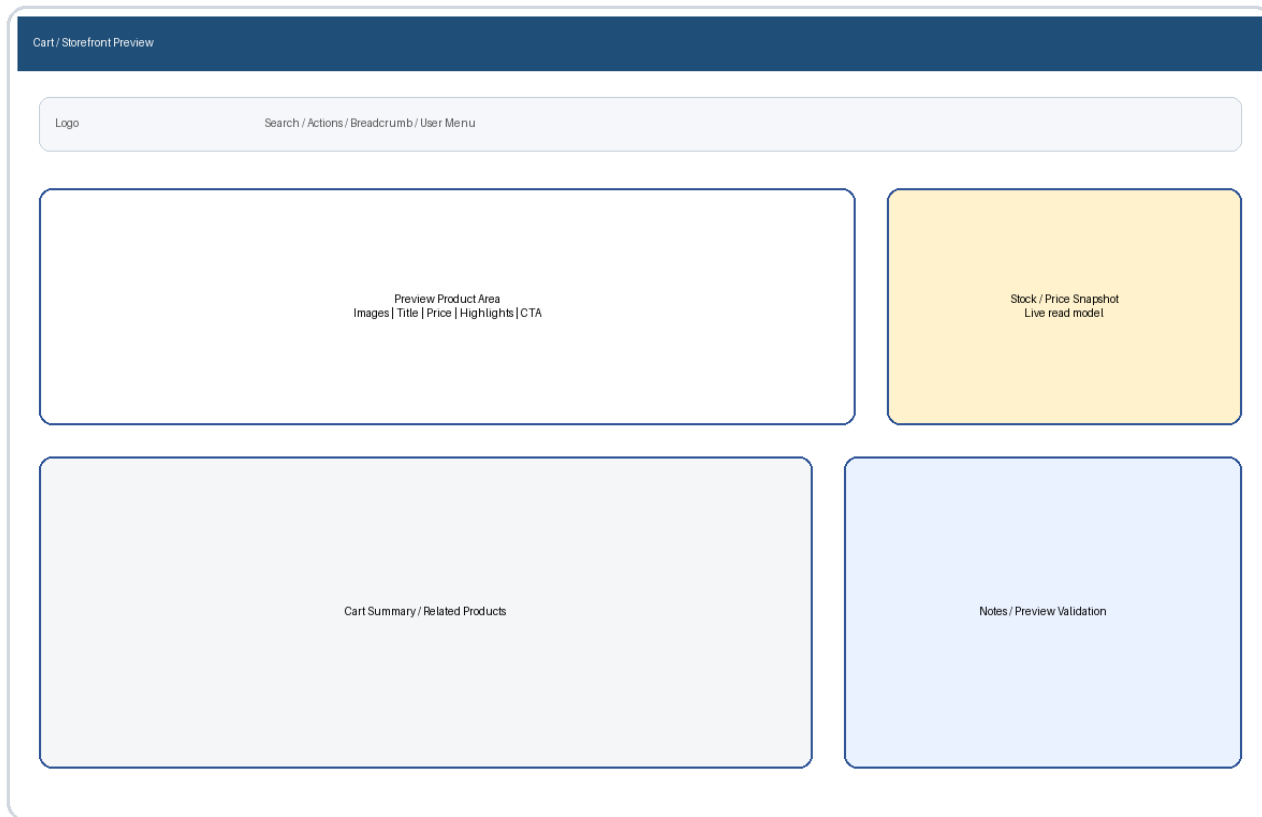
Pricing & Tax
MRP
Sale Price
GST
Date Range

Status & Publish
Draft / Review / Published

Wireframe reference - Product Details Edit

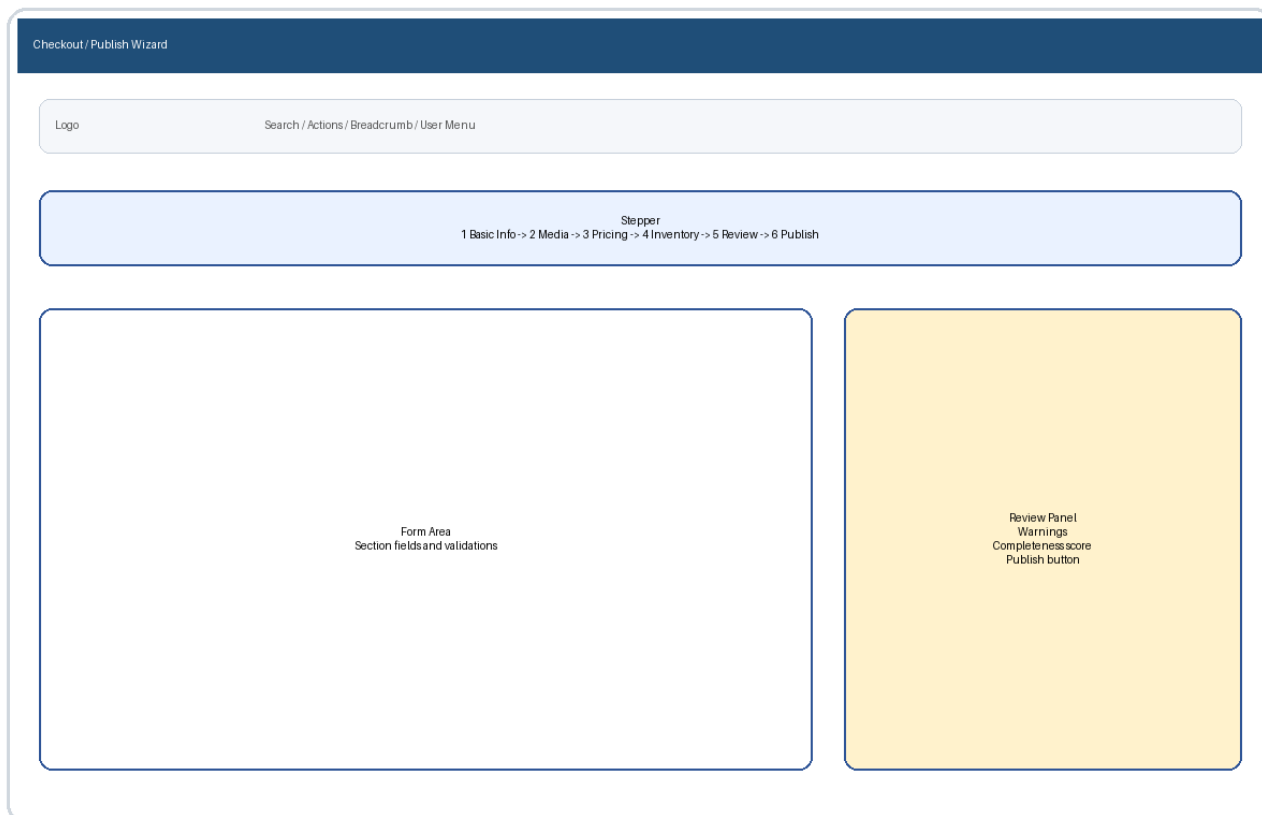
14

Cart Storefront Preview



Wireframe reference - Cart Storefront Preview

Checkout Publish Wizard



Order History Product Audit

Order History / Product Audit

Logo

Search / Actions / Breadcrumb / User Menu

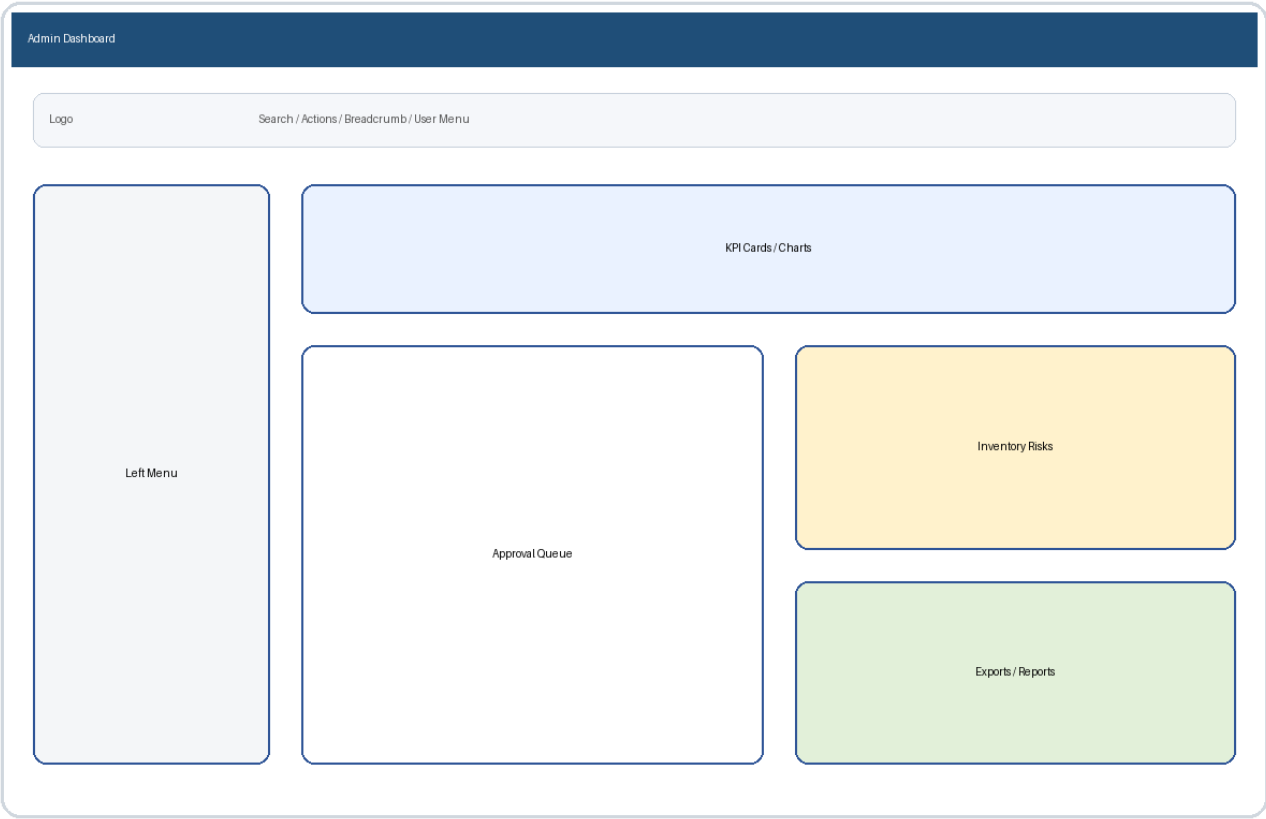
Search + Date Filters + Status Filters

Audit / History Table
Change | By | Time | Remarks

Selected Entry Detail
Before / After snapshot

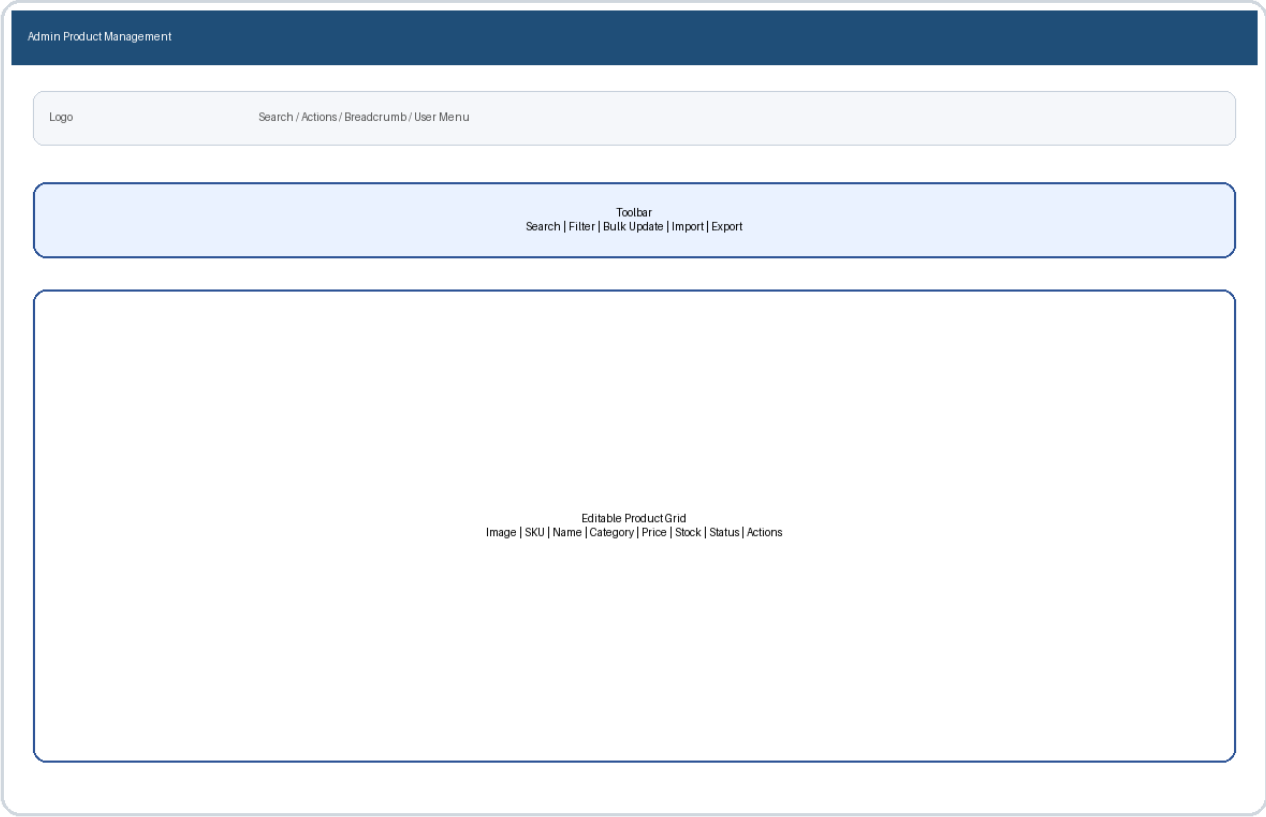
Wireframe reference - Order History Product Audit

Admin Dashboard

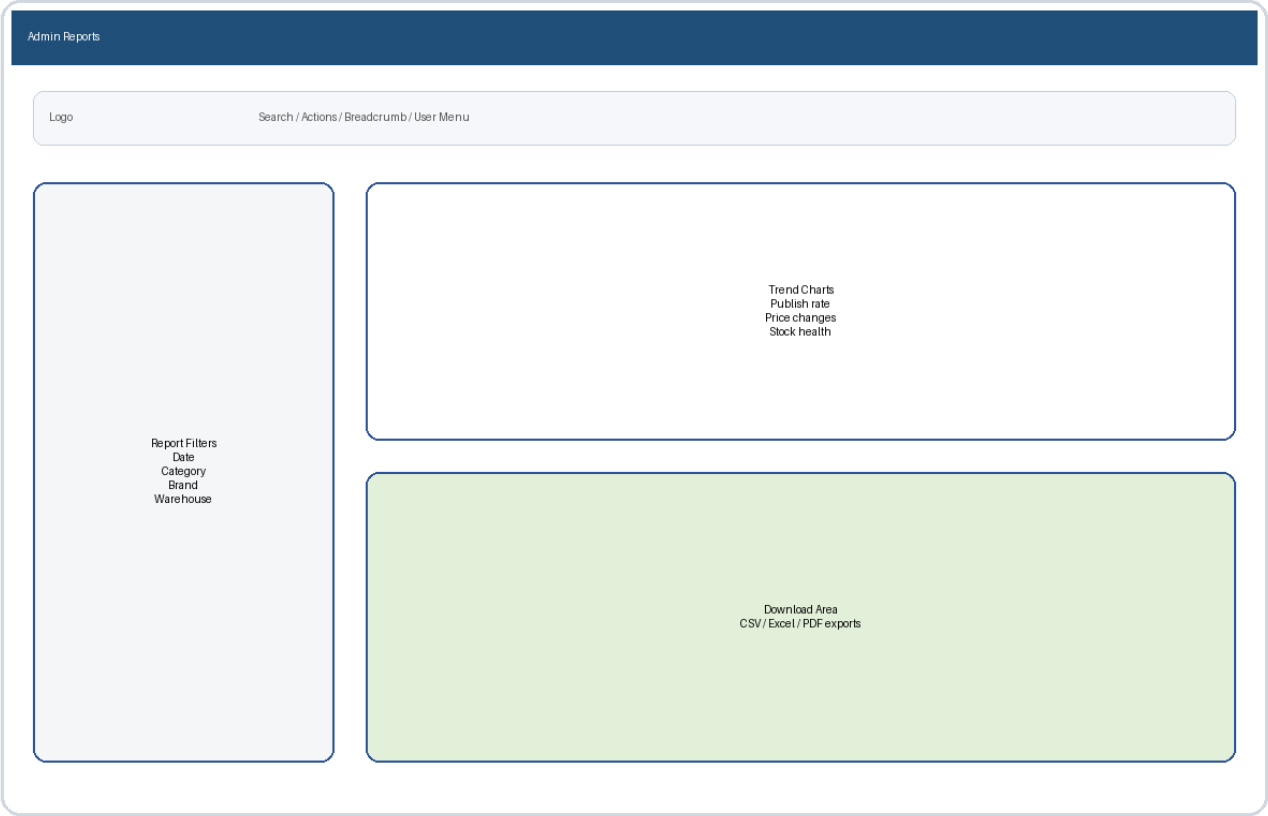


Wireframe reference - Admin Dashboard

Admin Product Management



Admin Reports



Wireframe reference - Admin Reports